

Email Availability Check

A single-purpose endpoint that tells the sign-up form whether an email address is already registered.

Endpoint: `POST /api/v1/auth/check-email` · **Auth:** none (public) · **Rate limit:** 10 / minute · **Date:** 2026-10-06

1 What it does

Given an email address, it answers exactly one question: **is this address already registered?** It is meant to be called from the sign-up form while the visitor is still typing — before they have any authentication token.

Method	POST
URL	<code>{BASE_URL}/api/v1/auth/check-email</code>
Authentication	None — deliberately public, because the visitor has no token yet
Rate limit	10 requests per minute, per IP. The 11th returns <code>429</code> .
Content-Type	<code>application/json</code>

2 Request

```
POST {BASE_URL}/api/v1/auth/check-email
Content-Type: application/json
Accept: application/json

{
  "email": "user@example.com"
}
```

Field	Type	Rules
<code>email</code>	string	Required · must be a valid email · max 255 characters

Case and whitespace are handled for you
The server lowercases and trims the address before looking it up — exactly the way registration does. So `"Ahmed@Example.com"` and `"ahmed@example.com"` get the same answer. You do not need to normalise it on the client, and you should not, so that both sides always agree.

3 Responses

200 — the address is well-formed (this is the normal case)

```
{
  "success": true,
  "message": "Email availability checked.",
  "data": {
    "email": "user@example.com",
    "exists": false,
    "available": true
  }
}
```

Field	Type	Meaning
<code>exists</code>	boolean	The primary field. <code>true</code> = already registered.
<code>available</code>	boolean	The inverse of <code>exists</code> , sent so you never have to negate it by hand.
<code>email</code>	string	The normalised address the server actually checked (echoed back).

Use `data.exists`

`exists: true` → show “this email is already registered”.

`exists: false` → the address is free.

422 — the address is malformed

```
{
  "message": "The email field must be a valid email address.",
  "errors": {
    "email": ["The email field must be a valid email address."]
  }
}
```

⚠️ 422 does NOT mean “already taken”

A `422` means the string you sent is not a usable email address at all — empty, missing, malformed, or longer than 255 characters. Treat it as “keep typing”, never as “this email is taken”. Note this response has no `success` key, unlike the 200.

429 — too many requests

More than 10 calls in a minute from the same IP. Back off and retry later; do not surface this to the user as a form error. See the recommended handling in section 5.

4 Guarantees you can rely on

Guarantee	Why it matters to you
It always agrees with registration	If this endpoint says <code>available: true</code> , then <code>POST /api/v1/auth/register</code> will succeed (201). If it says <code>exists: true</code> , registration will refuse the address (422). The two can never disagree — this is covered by automated tests that call the real registration endpoint.
Deleted accounts still count as registered	An account that was deleted still occupies its email address, and registration will still reject that address. This endpoint reports it as <code>exists: true</code> , so it will not promise you an address that registration then refuses.
No account details leak	The response contains only <code>email</code> , <code>exists</code> and <code>available</code> . There is no id, no name, no status, no role — nothing about the account behind the address. Do not expect more fields; they will not be added.

5 What it does *not* do

⚠ Disposable email domains are not checked

An address on a throwaway domain (e.g. `something@test.com`) returns `available: true` here, but **registration will still reject it** with a 422. The two questions — “is it registered?” and “is it a throwaway domain?” — are deliberately kept separate. **Consequence for you:** a green “available” result is not a guarantee that the address will be accepted at registration. Always let the real registration call be the final authority, and render its 422 error on the email field.

6 Recommended frontend usage

1. **Call it on `blur`, not on every keystroke** — debounce ~400 ms if you do listen to input.
2. **Cancel the previous request** when a new one starts, so an older response cannot overwrite a newer one.
3. **Fail open.** If the call fails (network error, `429`, `5xx`), let the user continue. Never block the form on this endpoint — registration is the source of truth.
4. **Never block submit on it.** Use the result as a hint; the registration response is what decides.
5. **Do not cache the answer for long.** Someone else may register that address a second later.

React example

```

import { useEffect, useState } from "react";

export function useEmailAvailability(email) {
  const [state, setState] = useState({ status: "idle" });

  useEffect(() => {
    const value = (email ?? "").trim();
    if (!value) { setState({ status: "idle" }); return; }

    // Debounce, and cancel a superseded request.
    const controller = new AbortController();
    const timer = setTimeout(async () => {
      setState({ status: "checking" });
      try {
        const res = await fetch(`${BASE_URL}/api/v1/auth/check-email`, {
          method: "POST",
          headers: { "Content-Type": "application/json", Accept: "application/json" },
          body: JSON.stringify({ email: value }),
          signal: controller.signal,
        });

        // 422 = malformed input, not "taken". Just stay quiet.
        if (res.status === 422) { setState({ status: "invalid" }); return; }

        // Fail open on throttling / server errors.
        if (!res.ok) { setState({ status: "unknown" }); return; }

        const { data } = await res.json();
        setState({ status: data.exists ? "taken" : "available" });
      } catch (err) {
        if (err.name !== "AbortError") setState({ status: "unknown" });
      }
    }, 400);

    return () => { clearTimeout(timer); controller.abort(); };
  }, [email]);

  return state;
}

```

How to render each state

State	Show	Block submit?
<code>available</code>	Neutral or a soft "looks good"	No
<code>taken</code>	"This email is already registered." Offer a sign-in link.	Optional — registration will refuse it anyway
<code>invalid</code> (422)	Nothing special — the normal client-side email check covers it	No
<code>unknown</code> (429 / 5xx / offline)	Nothing. Stay silent.	No — never punish the user for our outage

7 Quick test with cURL

```
curl -i -X POST "$BASE_URL/api/v1/auth/check-email" \  
-H "Content-Type: application/json" \  
-H "Accept: application/json" \  
-d '{"email":"someone@example.com"}'
```

A note on privacy (awareness only — no action needed)

An endpoint like this can be used to probe which addresses exist on the platform. It does not create a new exposure — `POST /api/v1/auth/register` already answers the same question — but it makes asking cheaper, so it is rate-limited to 10/minute and returns nothing beyond the boolean. If you display a “already registered” message, keep it matter-of-fact and always offer a sign-in link, rather than confirming anything about the account.